

Loan Rangers Milestone 4 Report

Problem Space:

Banks must be very careful when deciding on whether or not to approve bank loans. On one hand, if they are too cautious and refuse a loan, they may lose out on potential business gains. On the other hand, if a big loan is given out, and not repaid, it could cost the bank an immense amount of money. We aim to classify a loan application's risk and potential value. This includes determining whether or not approving the loan is a risk and then determining the benefit/detriment to the bank of approving the loan.

Approach:

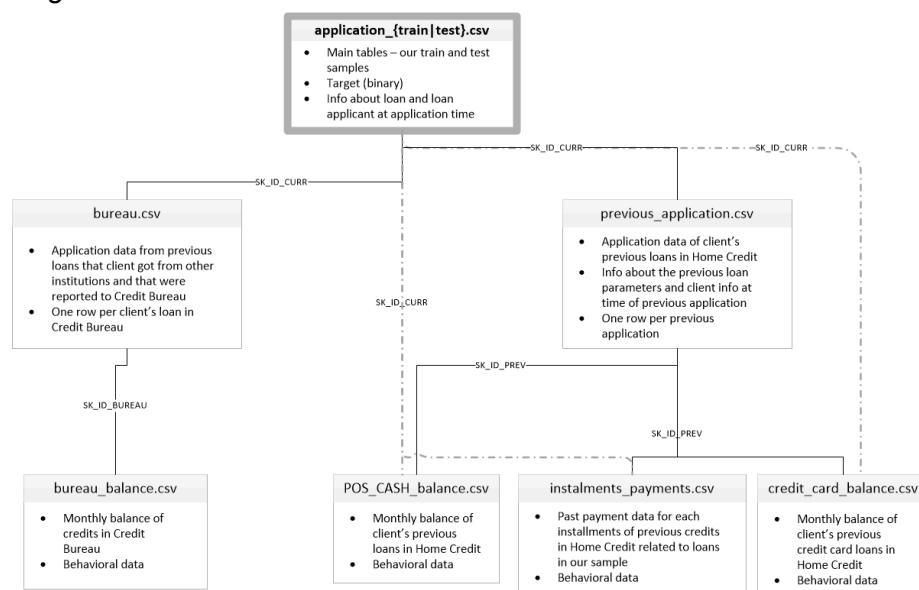
The use of applicant behavior and history to predict creditworthiness is not a new problem space; credit scores are a well-established tool to estimate creditworthiness. In addition, there is significant interest by credit institutions in applying machine learning models to this problem space to further refine the accuracy of predictions and improve business efficiency.

To that end, research has been done on applying both traditional ML methods such as Decision Trees, Naive Bayes, and Logistic Regression, and recent methods such as Deep Learning to further refine creditworthiness prediction accuracy. This represents a wealth of resources that our team can refer to during the implementation of our models, and also presents us the question: which model works best? This question forms the basis of our project.

Next, due to the sheer size and diversity of types of data (numerical continuous, numerical discrete, categorical, and ordinal) presented in the dataset, varied feature engineering techniques are required to fully harness the predictive power of the features in our dataset. The presence of both sequential and non-sequential data, as well as dynamic feature sizing further complicates our decision on what features to use, and how to engineer them effectively for use. Thus, the team plans to take an incremental approach toward incorporating features into the model.

The Dataset:

The dataset we are using comes from the [Home Credit Default Risk Kaggle Competition](#)¹ that was hosted 6 years ago.



The dataset comprises 7 tables. Each table (excluding the main application_train|test.csv table) has a one-to-many relationship with the table above it, which made generating a complete dataset challenging. For brevity, refer above for the contents of each table above.

Notably, the application_train|test.csv is an imbalanced dataset with 279864 negative samples and only 24667 positive samples. An example of the application_train|test.csv is shown below:

SK_ID_CURR	TARGET	NAME_CONTRACT_TYPE	CODE_GENDER	FLAG_OWN_CAR	FLAG_OWN_REALTY	CNT_CHILDREN	AMT_INCOME_TOTAL	AMT_CREDIT	AMT_ANNUITY	...	FLAG_DOCUMENT_18	FLAG_DOCUMENT_19	
0	100002	1	Cash loans	M	N	Y	0	202500.0	406597.5	24700.5	...	0	0
1	100003	0	Cash loans	F	N	N	0	270000.0	1293502.5	35698.5	...	0	0
2	100004	0	Revolving loans	M	Y	Y	0	67500.0	135000.0	6750.0	...	0	0
3	100006	0	Cash loans	F	N	Y	0	135000.0	312682.5	29686.5	...	0	0
4	100007	0	Cash loans	M	N	Y	0	121500.0	513000.0	21865.5	...	0	0
5 rows × 122 columns													

Data Cleaning:

Due to the lack of expert knowledge, as well as the presence of features with significant NaNs, we removed these features or encoded NaNs into suitable values. This, combined with removing rows with NaNs, allowed us to maintain most of the dataset.

Feature Encoding:

To capture the information in the auxiliary tables, we generated the mean, median, and variance of each group of samples by the parent sample to generate 3 variations of data:

1. application_train|test (Samples: 167912, Features: 250)
2. application_train|test + bureau (mean, med, var) + previous_application (mean, median var) (Samples: 167912, Features: 795)
3. all data, with child tables encoded into mean, median, and var (Samples: 167912, Features: 1266)

This approach was in the hopes of capturing the key information in the many samples related to the main sample.

Resampling:

To mitigate the effects of the imbalanced dataset, we tested random undersampling, [undersampling using Tomek's Links](#)^{2, 3}, and a combination of oversampling using [KMeans Synthetic Minority Oversampling Technique \(SMOTE\)](#)^{4, 5} with undersampling. This resulted in the following datasets:

Dataset/Dimensions	Tranche 1	Tranche 2	Tranche 3
Number of Features:	250	795	1266
Training	0.0: 167912, 1.0: 14806 Tot: 182718	0.0: 167912, 1.0: 14806 Tot: 182718	0.0: 167912, 1.0: 14806 Tot: 182718
KMeans SMOTE with Tomek Links	0.0: 167910, 1.0: 91891 Tot: 259801	0.0: 167909, 1.0: 94953 Tot: 259801	0.0: 167905, 1.0: 82803 Tot: 259801
Random Undersampling	0.0: 14806, 1.0: 14806	0.0: 14806, 1.0: 14806	0.0: 14806, 1.0: 14806

	Tot: 29612	Tot: 29612	Tot: 29612
Random Undersampling and KMeans SMOTE	0.0: 74030, 1.0: 14806 Tot: 88836	0.0: 74030, 1.0: 14958 Tot: 88988	0.0: 74030, 1.0: 15001 Tot: 89031
Test	0.0: 55976, 1.0: 4930 Tot: 60906	0.0: 55976, 1.0: 4930 Tot: 60906	0.0: 55976, 1.0: 4930 Tot: 60906
Validation	0.0: 55976, 1.0: 4931 Tot: 60907	0.0: 55976, 1.0: 4931 Tot: 60907	0.0: 55976, 1.0: 4931 Tot: 60907

Feature Selection:

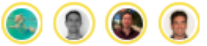
Due to the large number of features for the combined datasets, we used Principal Component Analysis (PCA) to extract the feature combinations that explained the most variance in the training set. The resulting orthogonal basis of the vector space generated removed all dimensions that could be expressed by a linear combination of other dimensions via the Gram-Schmidt Process. While this helped to reduce the number of features, it also reduced the explainability of the model.

Results:

To benchmark the performance of our models, we mainly use ROC-AUC:

1. That is the scoring metric used by the Kaggle Competition.
2. The ROC-AUC is independent of hyperparameters such as decision threshold.

Baseline 1: Kaggle Leaderboard

1	▲ 10	Home Aloan		0.80570	499	6y	
2	—	ikiri_DS		0.80561	477	6y	
3	▲ 1	alijs & Evgeny		0.80511	143	6y	
6136	▼ 26	RDCC		0.70502	4	6y	
6137	▼ 85	SMIN		0.70489	2	6y	
6138	▲ 111	adamren001		0.70486	4	6y	
6139	▼ 69	makers15		0.70482	17	6y	

Baseline 2: Preliminary Models

1. Random Forest Algorithm (With Gradient Boosting)

	Tranche 1	Tranche 2	Tranche 3
Training	0.65550	0.64966	0.61137
KMeans SMOTE with Tomek Links	0.62566 0.62880	0.62694 0.63794	0.63791
Random Undersampling	0.65537	0.64966	0.61137
Random Undersampling and KMeans SMOTE	<u>0.71708</u> 0.72400	0.67342	0.61361

HyperParameter-Optimized Score is highlighted

2. Simple Neural Network (2 layers, upside-down-pyramid-shaped)

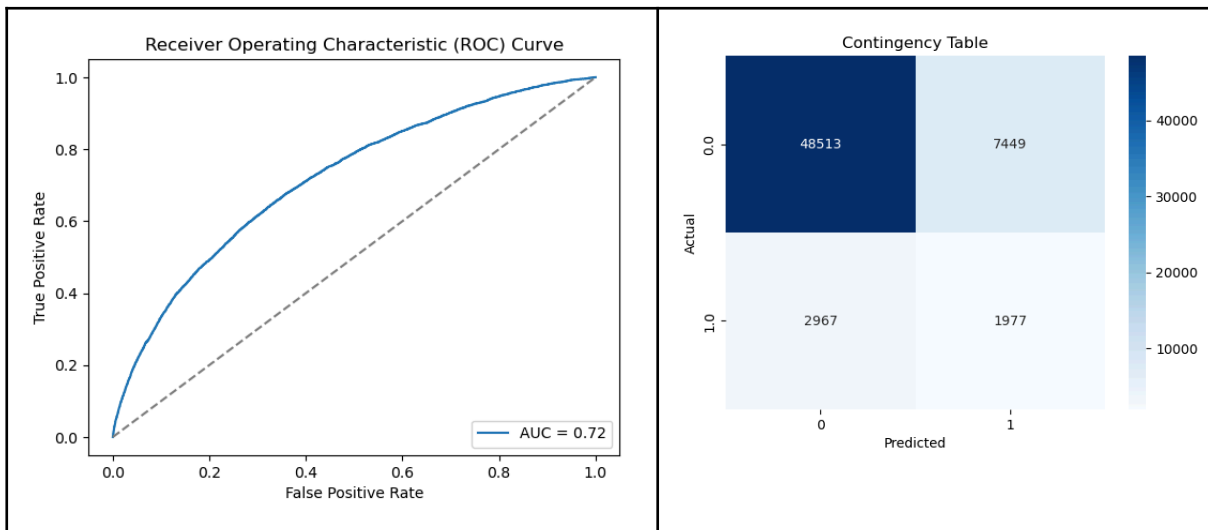
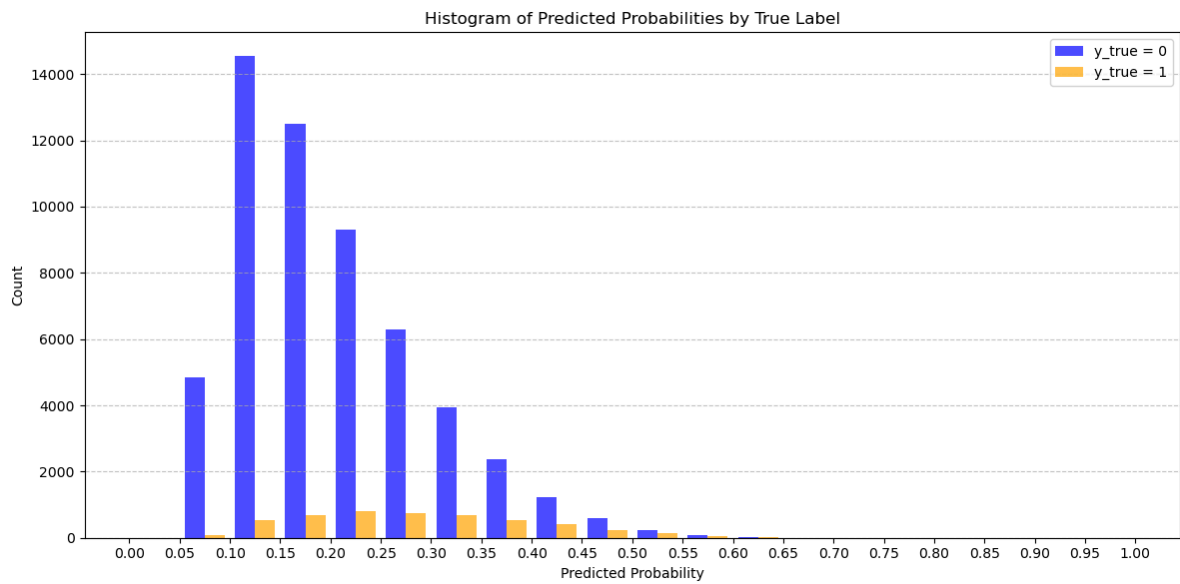
	Tranche 1	Tranche 2	Tranche 3
Training	0.69650	0.69843	0.70503
KMeans SMOTE with Tomek Links	0.65663	0.66357	0.64474
Random Undersampling	0.69232	0.68897	0.70134
Random Undersampling and KMeans SMOTE	0.69404	<u>0.70722</u>	0.70098

Best Baseline Results:

1. Random Forest with Random Undersampling and KMeans SMOTE on Tranche 1

To generate this dataset, a KMeans clustering analysis was performed to find the number of clusters (3 for Tranche 1) that reduced the distances between points to clusters using the “elbow method”. The dataset was then undersampled randomly such that the ratio of Minority/Majority classes was 0.2. The resulting dataset was then separated into 3 clusters, and SMOTE was applied to each cluster with a ratio of Majority/Minority classes less than 3.0. In this case, there were no suitable clusters and the final dataset was just a random undersampling.

1. F1 score (at threshold 0.31): 0.27515657620041756
2. ROC-AUC: 0.71708 (#6012 on Kaggle Leaderboard)



(Results using baseline hyperparameters)

Following this, we performed a hyperparameter search to try and improve our results further. Due to the time taken for each hyperparameter search, we limited our scope to the following combinations of models and datasets:

1. Dataset: Tranche 1, Random Undersampling and KMeans SMOTE;
Model: Random Forest Algorithm (With Gradient Boosting)
ROC-AUC: 0.72400 (+0.00692)
2. Dataset: Tranche 1, KMeans SMOTE with Tomek Links;
Model: Random Forest Algorithm (With Gradient Boosting)
ROC-AUC: 0.62880 (+0.00314)
3. Dataset: Tranche 2, KMeans SMOTE with Tomek Links;
Model: Random Forest Algorithm (With Gradient Boosting)
ROC-AUC: 0.63794 (+0.011)

Our decision not to perform a hyperparameter search on the Neural Networks was due to the fact that it takes a much longer time for each model to converge than the Random Forest models.

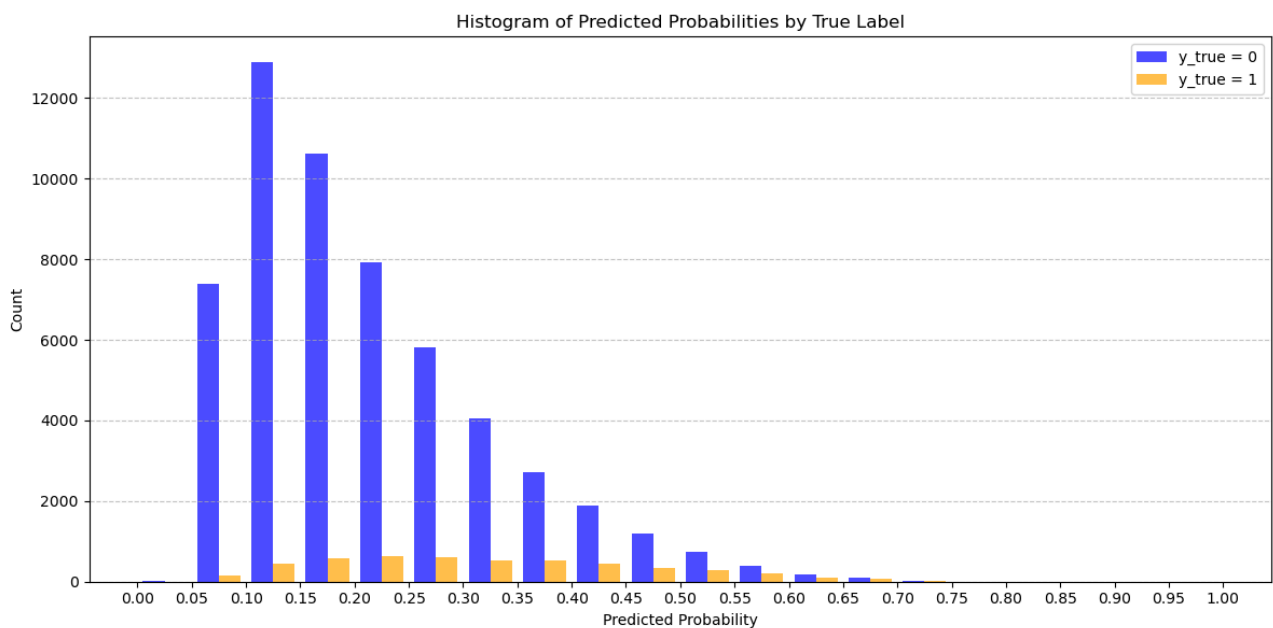
Best Results After Hyperparameter Search:

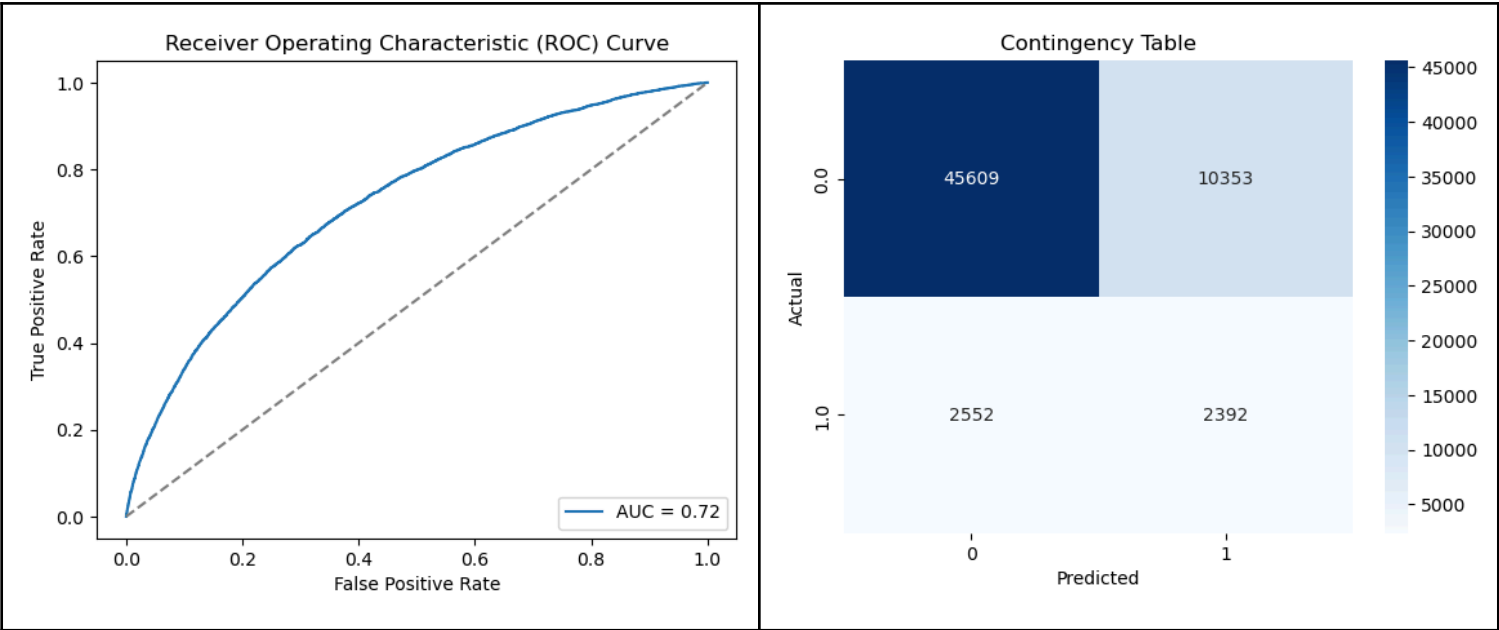
For brevity, we only present the results of the grid search performed on the best performing baseline model.

Hyperparameter	Original Values	Best Values	Search Range
max_depth	31	44	5-50
num_leaves	Default value	54	20-150
learning_rate	0.01	0.012254	0.001-1.0, log
n_estimators	1000	872	750-1000, log
reg_alpha	Default value	0.000271	0.0001-10.0, log
reg_lambda	0.5	0.325	0.1-1.0
scale_pos_weight	1.4	1.48	1-2
n_components_ratio	1.0	0.999	0.95,0.99, 0.999, 1

Notably, we only gained a marginal increase in ROC-AUC scores, which aligns with the broader trends across the 2 other searches. This, coupled with best-in-class results from the Kaggle leaderboard of 0.80570 suggest that the other avenues such as improved data encoding and feature transformation are required to improve class separability.

1. F1 score (at threshold 0.351): 0.2798751064433721
2. ROC-AUC: 0.72400 (#5932 on Kaggle Leaderboard)





(Model results using best hyperparameters)

Further Discussion and Future Directions

This work offers a preliminary exploration into modeling credit risk and evaluating loan approvals using a complex dataset. While the initial results are encouraging, several aspects warrant deeper investigation and refinement.

Feature Engineering and Data Representation

One central challenge is effectively integrating various data types—continuous, discrete, categorical, ordinal, and sequential—into a cohesive predictive model. Our current strategy, which included aggregating subsidiary tables through summary statistics and applying PCA for dimensionality reduction, may have constrained the model’s ability to detect subtle patterns and time-dependent structures. Future efforts could employ automated feature engineering methods (e.g., feature importance-driven selection or penalization) and advanced encoding strategies that preserve temporal relationships and domain-specific patterns, potentially enhancing predictive accuracy.

Model Selection and Explainability

While our initial models established a baseline, more sophisticated approaches could better leverage the dataset’s complexity. Specialized deep learning architectures designed for mixed and sequential data may yield improved performance. Additionally, explainability tools like SHAP or LIME can clarify which features drive predictions, increasing transparency and trust—crucial factors in banking contexts that require both predictive strength and interpretability.

Revisiting Resampling Strategies

Although we experimented with undersampling, SMOTE, and Tomek Links, data imbalance remains a substantial hurdle. Exploring more nuanced oversampling techniques or integrating cost-sensitive learning approaches may better address skewed class distributions. Iterative testing and careful validation will be critical to ensure that any resampling method does not inflate performance metrics or produce overly optimistic conclusions.

Hyperparameter Optimization and Computational Resources

Hyperparameter tuning led to incremental gains but required extensive computational effort. Employing more efficient search techniques, such as Bayesian optimization or evolutionary algorithms, could streamline this process. Additional computational resources—parallelization, and cloud-based infrastructure—would broaden the search space, potentially uncovering more optimal model configurations.

Monetary Risk Modeling

While predicting loan approvals is useful, incorporating financial metrics would provide a more comprehensive decision-making framework. Estimating expected returns, losses, and profit margins would align predictive risk scores with economic outcomes, producing a richer, more actionable model. Though deferred due to complexity and resource constraints, this step remains an important direction for future research.

Conclusion

This initial work lays a foundation for predicting credit risk and loan approvals but also highlights several avenues for enhancement. Through improved feature engineering, advanced modeling techniques, more thoughtful resampling strategies, efficient hyperparameter optimization, and the integration of financial considerations, subsequent iterations can deliver models that are both more predictive and more aligned with the practical needs of banking decision-makers.

- [1] Home Credit, “Home Credit Default Risk,” *kaggle.com*, 2018. Available: <https://www.kaggle.com/competitions/home-credit-default-risk/data>
- [2] Imblearn, “SMOTETomek — Version 0.12.4,” *Imbalanced-learn.org*, 2014. Available: <https://imbalanced-learn.org/stable/references/generated/imblearn.combine.SMOTETomek.html#id1>. [Accessed: Dec. 15, 2024]
- [3] Gustavo, Ana, and M. C. Monard, “Balancing Training Data for Automated Annotation of Keywords: a Case Study,” pp. 10–18, Jan. 2003.
- [4] Imblearn, “KMeansSMOTE — Version 0.12.4,” *Imbalanced-learn.org*, 2014. Available: https://imbalanced-learn.org/stable/references/generated/imblearn.over_sampling.KMeansSMOTE.html. [Accessed: Dec. 15, 2024]
- [5] G. Douzas, F. Bacao, and F. Last, “Improving imbalanced learning through a heuristic oversampling method based on k-means and SMOTE,” *Information Sciences*, vol. 465, pp. 1–20, Oct. 2018, doi: <https://doi.org/10.1016/j.ins.2018.06.056>